spark / GAME DESIGNER

MISSION 02

Build a player that moves

Start in one file. Give your player its own state, then share your scene helpers.

01 PREDICT 02 TRACE 03 EXPLAIN

PLAY · THINK · CODE · CREATE · GROW

Concept art / Explore what the code does



One file. A visible game loop.

Import gives us tools; our loop decides when to use them.

Read the code

```
import pygame
while running:
    keys = pygame.key.get_pressed()
    pygame.display.flip()
```

01
Get tools
import pygame

02
Read and update
keys x, y



03
Show frame
display.flip()



Your prediction: Does import pygame move the robot?

Let’s check: No. The loop reads input, updates position and draws each frame.

Reference: 01_loop.py / Illustrated trace / Selected lines are not standalone

One tiny move, then another

The loop caps dt at 0.05 seconds. speed is pixels per second.

Read the code

```
dt = min(clock.tick(60) / 1000, 0.05)
x += dx * speed * dt
y += dy * speed * dt
```

Start

x = 160

Right: one update

1 × 180 × 0.05

x = 169

Right: next update

x = 178

Your prediction: Release the key: dx=0. What changes next?

Let’s check: Position stays 178; the loop still draws.

Give your drawing a name

A helper is a function for a named job. Keep it in main.py first.

Read the code

```
def draw_scene(screen, pad, pad_position):
    screen.fill("#DDF2FF")
    pygame.draw.rect(screen, "#95C549", (0, 420, 800, 180))
    pygame.draw.rect(screen, "#F7D34B", (80, 390, 640, 18))
    screen.blit(pad, pad.get_rect(center=pad_position))
```

01
Clear
sky colour

→

02
Build the scene
ground + route + landing pad



→

03
Draw the player last
robot on top



Your prediction: What if we clear the screen after drawing the player?

Let’s check: It erases the player. Clear first; draw the background, pad and then player.

Meet the Player blueprint

The class describes a player. Calling Player(...) creates an instance.

Read the code

```
class Player:
    def __init__(self, x, y, image):
        self.x = x
        self.y = y
        self.speed = 180
        self.image = image
player = Player(160, 300, robot_image)
```

01

Class

Player

state + methods

02

Instance

player

x = 160

y = 300

speed = 180



Your prediction: Is the image the whole player object?

Let’s check: No. The instance also owns position and speed, and uses move/draw methods.

Reference: main.py / Illustrated trace / Selected lines are not standalone

self means this player

A method is a function on an object. The call chooses the receiver.

Read the code

```
def move(self, dx, dy, dt):
    self.x += dx * self.speed * dt
    self.y += dy * self.speed * dt
    self.x = max(32, min(768, self.x))
    self.y = max(32, min(568, self.y))
player.move(dx, dy, dt)
```

01
Before
player.x = 160



02
Method call
dx=1, dy=0
dt=0.05
self = player

03
After
player.x = 169



Your prediction: Does player.draw(screen) change x?

Let’s check: No. move changes state; draw reads that state to place the image.

Reference: main.py / Illustrated trace / Selected lines are not standalone

Two players, two positions

This position-only probe uses None because it does not draw an image.

Read the code

```
nova = Player(160, 300, None)
pip = Player(500, 300, None)
nova.move(1, 0, 0.05)
print(nova.x, pip.x)
```

01
Before

nova.x = 160
pip.x = 500

02
Move Nova only

nova.x = 169



03
Pip stays put

pip.x = 500



Your prediction: Would creating Pip reset Nova?

Let’s check: No. Each instance owns its attributes. self is Nova only during nova.move.

Same route. A different speed.

Keep the start and target fixed for a fair comparison.

Read the code

```
self.speed = 240
self.x += dx * self.speed * dt
```

01

Baseline

180 × 0.05

9 pixels



02

Modified

240 × 0.05

12 pixels



Your prediction: Which speed helps you stop on the pad? Test your prediction.

Let’s check: Preferences may differ. Use an observed docking attempt to explain the trade-off.

Reference: teacher_example.py / Illustrated trace / Selected lines are not standalone

Your helper can travel

Move the two helper definitions to scene.py. Keep the loop and Player in main.py.

Read the code

```
from scene import load_sprite, draw_scene
robot_image = load_sprite(images / "robot.png", 64)
draw_scene(screen, pad, pad_position)
player.draw(screen)
```

01 Define

scene.py
load_sprite
draw_scene

→

02 Import

main.py
from scene
import …

→

03 Call

once: load_sprite
each frame:
draw_scene

Your prediction: Does importing draw_scene draw a frame?

Let’s check: No. Import makes the helper available. Its call inside the loop draws the scene.

Reference: modular/main.py / Illustrated trace / Selected lines are not standalone

Sprite lab · four ways to face

One robot, four pictures. Camera above the floor. Direction frames are not yet a walk animation.

Read the code

```
    "up": load_sprite(images / "robot_up.png", 64),
    "right": load_sprite(images / "robot_right.png", 64),
    "down": load_sprite(images / "robot_down.png",
64),
    "left": load_sprite(images / "robot_left.png", 64),
```

01
Up
up



02
Right
right



03
Down
down



04
Left
left



Your prediction: Which picture shows the backpack most clearly?

Let’s check: Up: we see the back. A sprite is a game image; Player holds its position and behaviour.

Reference: 04_sprites.py / Illustrated trace / Selected lines are not standalone

A wardrobe of pictures

A dictionary pairs each direction key with an image. Load once, before the loop.

Read the code

```
self.sprites = sprites
self.facing = "down"
image = self.sprites[self.facing]
```

01
Key
"down"

→

02
Image value
sprites["down"]



→

03
Selected frame
image



Your prediction: Is the string "down" itself a picture?

Let’s check: No. The string is a key; the dictionary lookup returns the loaded image.

Input chooses facing

Keep this chain inside Player.move. Vertical input wins diagonals.

Read the code

```
if dy < 0:
    self.facing = "up"
elif dy > 0:
    self.facing = "down"
elif dx < 0:
    self.facing = "left"
elif dx > 0:
    self.facing = "right"
```

01
Up + Right
dx=1, dy=-1

→

02
First true branch
dy < 0

→

03
Face up
facing="up"



Your prediction: What changes if you put dx branches first?

Let’s check: The picture faces right. Position still moves diagonally because both x/y updates remain.

Stop moving. Keep looking!

No matching branch changes facing when both inputs are zero.

Read the code

```
        self.x += dx * self.speed * dt
        self.y += dy * self.speed * dt
        image = self.sprites[self.facing]
        rectangle = image.get_rect(center=(round(self.x),
round(self.y)))
        screen.blit(image, rectangle)
```

01
Hold Right
right



02
Release
right



03
Hold Up
up



Your prediction: Would else: facing="down" keep this memory?

Let’s check: No. It would face down on release. Choosing a picture and changing position are separate jobs.

Two players, two directions

Each object keeps its own facing. They may share the image library.

Read the code

```
def __init__(self, x, y, sprites):
    self.x = x
    self.y = y
    self.speed = 180
    self.sprites = sprites
    self.facing = "down"
```

01

Only a moves left

a: left



02

Then a stops

a: left



03

b was not moved

b: down



Your prediction: Which attribute is different, even with the same sprites dictionary?

Let’s check: self.facing. Changing a.facing does not change b.facing. Trace a new input for each object.